

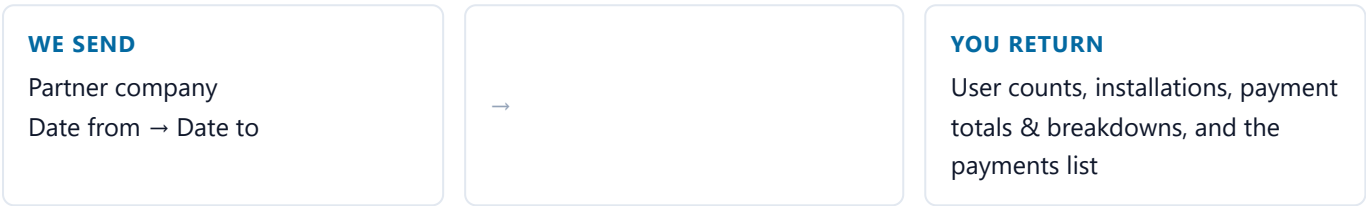
Partner App API — Data Requirements

What the CRM needs from the mobile-app backend

Version 1.1 · 17 July 2026 · For: Backend team (Django)

1. Overview

Qonshu CRM shows an **Analytics** view for every partner company that uses the mobile app. To populate it, the CRM sends a **partner** and a **date range**, and expects that partner's numbers for that range in return.



Read-only. The CRM never writes to your system. One endpoint is enough.

2. Request

```
GET /api/qonshu/partner-stats?partner=Acme+Towers&date_from=2026-07-01&date_to=2026-07-31
```

Parameter	Example	Notes
partner	Acme Towers	The partner company name (agreed: we match by name, not id). We enter the name on our side manually to match yours.
date_from	2026-07-01	Start of the range (inclusive).
date_to	2026-07-31	End of the range (inclusive).

Important — names must be stable and exact. Because we match on the company name, please treat it as a key: don't rename a partner without telling us, and keep spelling/spacing consistent. If a name changes on your side, that partner's analytics go empty in the CRM until we update it. Matching should be case-insensitive and ignore leading/trailing spaces. If a partner name is unknown, return `404` so we can spot the mismatch rather than silently show zeros.

3. Response

A. Users

Field	Type	Meaning
total_users	number	All app users of this partner.
active_users	number	Users currently active in your system.
engaged_users	number	Users who made at least one payment within the date range.
total_debt	number	Total amount currently owed by this partner's users.

B. Installations

Field	Type	Meaning
total_installations	number	How many users installed the app.
ios_installations	number	Installed on iOS.
android_installations	number	Installed on Android.
logged_in_users	number	Number of users who have an app token — i.e. they installed <i>and</i> logged in. This is how we measure real usage, not just downloads. <i>(Send the count only — we don't need the token values.)</i>

C. Payments in the date range

Field	Type	Meaning
total_payments_count	number	How many payments were made.
total_payments_amount	number	Total money paid.
amount_by_card	number	Of that total — paid by card.
amount_by_manual	number	Of that total — recorded manually.
amount_by_cash	number	Of that total — paid in cash.
utility_payments_count	number	How many utility payments.
utility_payments_amount	number	Total utility amount.
apartment_payments_amount	number	Total for apartments.
parking_payments_amount	number	Total for parking.
non_residential_payments_amount	number	Total for non-residential units.

The three **method** amounts (card + manual + cash) should add up to `total_payments_amount`. The four **category** amounts (utility + apartment + parking + non-residential) should also add up to it.

D. Payments list

The individual payments in the range, so the CRM can show a transactions table.

Field	Type	Meaning
id	string	Your id for the payment.
user_id	string	Who paid.
user_name	string	Their name (or a label, if names can't be shared).
date	datetime	When the payment happened.
amount	number	How much.
method	enum	CARD · MANUAL · CASH
category	enum	APARTMENT · PARKING · NON_RESIDENTIAL · UTILITY

4. Example

```
{
  "partner": "Acme Towers",
  "date_from": "2026-07-01",
  "date_to": "2026-07-31",

  "total_users": 1342,
  "active_users": 1180,
  "engaged_users": 640,
  "total_debt": 18420.75,

  "total_installations": 1200,
  "ios_installations": 520,
  "android_installations": 680,
  "logged_in_users": 900,

  "total_payments_count": 3120,
  "total_payments_amount": 154300.50,
  "amount_by_card": 120400.00,
  "amount_by_manual": 21900.50,
  "amount_by_cash": 12000.00,

  "utility_payments_count": 810,
  "utility_payments_amount": 39200.00,
  "apartment_payments_amount": 82100.50,
  "parking_payments_amount": 21000.00,
  "non_residential_payments_amount": 12000.00,

  "payments": [
    {
      "id": "pay_558210",
      "user_id": "usr_10293",
      "user_name": "Aysel Mammadova",
      "date": "2026-07-14T09:21:00Z",
      "amount": 149.90,
      "method": "CARD",
      "category": "UTILITY"
    }
  ]
}
```

5. Format notes

Topic	Rule
Dates	ISO format — <code>2026-07-01</code> for the range, <code>2026-07-14T09:21:00Z</code> for timestamps.
Money	Number with 2 decimals — <code>149.90</code> . Always positive (no refunds/negative amounts).
Cancelled payments	Exclude them. Cancellations are handled on your side — a cancelled payment must not appear in <code>payments</code> and must not be counted in any total.
No data	Return the same structure with <code>0</code> s and an empty <code>payments</code> list — not an error.
Unknown partner	Return <code>404</code> (so a name mismatch is visible instead of silently showing zeros).
Long lists	If <code>payments</code> is very long, paginate it — but keep the totals complete for the whole range.
Repeat calls	We call this endpoint repeatedly and always use the latest response, so cancellations and corrections on your side appear in the CRM automatically. Please keep it read-only and safe to call often.

6. Agreed decisions

Question	Decision
Payment methods	Confirmed — <code>CARD</code> , <code>MANUAL</code> , <code>CASH</code> match your system.
Payment categories	Confirmed — <code>APARTMENT</code> , <code>PARKING</code> , <code>NON_RESIDENTIAL</code> , <code>UTILITY</code> .
Logged-in users	Yes — you provide the count of users holding an app token; that is our <code>logged_in_users</code> .
Refunds	None exist — no negative amounts.
Cancelled payments	Handled on your side ; excluded from the response. Our numbers update automatically on the next call.
Partner identifier	Company name (no id). Names entered manually on our side — please keep them stable and exact.

7. Still needed from you

1. The **base URL** (staging + production) and how we should **authenticate** (e.g. a token we send in a header).
2. The **exact list of partner company names** as they appear in your system, so we can enter them on our side without typos.
3. A **sample response** from staging for one real partner (even a few rows), so we can validate against real data before go-live.